

Penetration Testing Report

Project Name: Lingaro Icons Catalog

Author: Radoslaw Maczuga (IT Security)

Date: 2026-04-13

Version: 1.0 (Initial Report)

Target: <https://github.com/Lingaro/Lingaro-icons-catalog>

EXECUTIVE SUMMARY

This document presents the findings of the penetration testing engagement conducted on the Lingaro Icons Catalog application. The assessment focused on the security of authentication mechanisms, authorization controls, file handling, input validation, and client-side attack surface.

The assessment identified multiple significant security vulnerabilities spanning authentication misconfiguration, missing access controls on file serving endpoints, unsanitized file path handling in the import pipeline, and a confirmed Stored XSS vector via SVG file upload. The most impactful finding is a vulnerability chain consisting of fail-open authentication combined with path traversal in the import endpoint and an unauthenticated file serving endpoint, which allows a completely unauthenticated attacker to read arbitrary files from the server filesystem - including source code, secrets, and the entire SQLite database - without any credentials.

The overall security posture is assessed as CRITICAL. The LIC-01 + LIC-03 chain achieves a CVSS score of 10.0 and was fully demonstrated with a working proof-of-concept script.

Finding Summary

ID	Vulnerability	Severity	Time Effort	Status
LIC-01	Fail-Open Authentication (Default Unauthenticated Mode)	CRITICAL	Low	Open
LIC-02	Unauthenticated File Serving Endpoint	CRITICAL	Low	Open
LIC-03	Arbitrary File Read via Import Path Injection + File Serving Chain	HIGH	Low	Open
LIC-04	Zip Slip in Azure Icon Import	HIGH	Medium	Open
LIC-05	Insecure Outbound HTTP Request - Missing Redirect Validation and Timeout	MEDIUM	Medium	Open
LIC-06	Stored XSS via Extension-Only File Type Validation	MEDIUM	Low	Open

1. ABOUT THE DOCUMENT

1.1 Document Objective

The objective of this document is to detail the security vulnerabilities identified during the assessment of the Lingaro Icons Catalog application. It provides technical descriptions of the findings, their potential business impact, and specific recommendations for remediation to improve the security posture of the application.

1.2 Version History

Version	Date	Author	Description
1.0	2026-04-14	Radoslaw Maczuga	Initial Report Release

1.3 Scope

The scope of the assessment included:

- Authentication & Authorization: fail-open logic, missing `require_auth` on endpoints
- Input Validation: file path sanitization, file type validation, FTS5 query construction
- File Handling: path traversal in import and upload endpoints, zip slip
- Client-Side Security: Stored XSS via SVG file serving
- Infrastructure Exposure: SSRF in Azure icon importer script
- Source Code Review: `api/dependencies.py`, `api/routers/icons.py`, `api/services/storage.py`, `scripts/import_azure_icons.py`

1.4 Limitations

The assessment was conducted on a staging environment running inside a Docker container. The environment did not contain production user data, limiting the ability to fully demonstrate the impact of certain findings. SSRF testing was constrained by the hardcoded URL in `import_azure_icons.py`, which prevents direct exploitation via the HTTP endpoint.

1.5 Methodology

Source Code Security Review

A manual review of the application source code was conducted to identify insecure coding practices, focusing on authentication logic, input validation, file path handling, and storage backend implementation.

Dynamic Testing

Vulnerabilities were tested dynamically using `curl` and custom Python scripts to verify exploitability and develop proof-of-concept exploits against a locally deployed Docker instance.

Multipart Parsing Analysis

Starlette's multipart/form-data parser was tested for filename handling quirks to identify potential extension validation bypasses.

1.6 Severity Scoring

Findings are rated based on the Common Vulnerability Scoring System (CVSS) v3.1 framework:

- Critical: CVSS 9.0 - 10.0
- High: CVSS 7.0 - 8.9
- Medium: CVSS 4.0 - 6.9
- Low: CVSS 0.1 - 3.9

2. FINDINGS

LIC-01	Fail-Open Authentication (Default Unauthenticated Mode)	CRITICAL	Open
File	api/dependencies.py:91-92		

CVSS v3.1 Score: 9.8 - AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H

Description of the Issue

Authentication is enabled conditionally - only when the environment variables AZURE_CLIENT_ID and API_KEY are both set. The application ships without any default values for these variables, meaning the server starts with no authentication if they are not explicitly configured. In this fail-open state, every incoming request - including requests to administrative routes - automatically receives full is_admin=True privileges. The _check_admin() function at line 119 contains a hardcoded pass-through for the "dev-mode" email value, which eliminates any secondary protection layer.

```
def _resolve_user(...) -> CurrentUser:
    if not os.getenv("AZURE_CLIENT_ID") and not os.getenv("API_KEY"):
        return CurrentUser(email="dev-mode", name="Developer", is_admin=True,
                             auth_method="api_key")
```

Impact

If the application is deployed without configured environment variables - whether due to misconfiguration, a failed secrets injection in CI/CD, or a developer test deployment - the server exposes full administrative access to all endpoints without requiring any credentials. All admin routes including import, delete, and refresh operations are wide open to any network-accessible client.

Evidence

A simple unauthenticated request returns full administrator privileges:

```
curl -s http://localhost:8000/api/me
```

```
{"email": "dev-mode", "name": "Developer", "is_admin": true}
```

Recommendation

The server should refuse to start if authentication credentials are not configured. Implement a startup check that raises a fatal exception rather than logging a warning and continuing. Authentication must be fail-safe, not fail-open.

LIC-02	Unauthenticated File Serving Endpoint	CRITICAL	Open
File	api/routers/icons.py:159-174		

CVSS v3.1 Score: 8.6 - AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N

Description of the Issue

The GET `/[icon_id]/file` endpoint does not include the `require_auth` dependency in its function signature, while every other icon endpoint does. This means the file serving endpoint remains fully public and accessible without any credentials, regardless of how authentication is configured at the application level.

```
@router.get("/{icon_id}/file")
async def get_icon_file(
    icon_id: str, db=Depends(get_database), storage=Depends(get_storage_backend),
): # missing: user=Depends(require_auth)
```

Impact

The file serving endpoint is permanently public, independent of authentication configuration. Even if LIC-01 is remediated and proper authentication is enforced across the application, this endpoint remains accessible without credentials. Combined with LIC-03, it forms a complete unauthenticated file exfiltration chain that persists after partial remediation.

Evidence

Direct invocation without an authorization token returns file contents with HTTP 200:

```
curl -sv http://localhost:8000/api/icons/test_icon_id/file
< HTTP/1.1 200 OK
< content-type: image/svg+xml
```

Recommendation

Add `user=Depends(require_auth)` to the `get_icon_file` function signature, consistent with all other icon endpoints. Conduct a review of all router files to verify consistent application of authentication dependencies across every endpoint.

LIC-03	Arbitrary File Read via Import Path Injection + File Serving Chain	HIGH	Open
File	api/routers/admin.py:36-46, api/services/storage.py:41-44, api/routers/icons.py:166		

CVSS v3.1 Score: 9.1 (with LIC-01) - AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N

RESULT: VULNERABLE - arbitrary file read confirmed with working PoC

Description of the Issue

The /api/admin/import endpoint accepts arbitrary JSON containing icon records and inserts the path field directly into the SQLite database without any sanitization or validation. When GET /api/icons/{id}/file is subsequently called, the stored path is passed directly to LocalStorage.get() through pathlib without normalization.

```
# Step 1: path field stored without validation
storage_key = icon["path"].removeprefix("icons/") # no sanitization
data = storage.get(storage_key)

# Step 2: LocalStorage.get()
path = self.base_dir / key # pathlib: base_dir / "/etc/passwd" = /etc/passwd
```

The bypass mechanism relies on pathlib's behavior when the right-hand operand is an absolute path: Python's pathlib discards base_dir entirely. Injecting "icons//etc/passwd" as the path value causes removeprefix("icons/") to yield "/etc/passwd" - an absolute path - which pathlib resolves directly, ignoring base_dir.

Attack Chain

- POST /api/admin/import with "path": "icons//etc/passwd" (succeeds due to LIC-01)
- removeprefix("icons/") yields "/etc/passwd" (absolute path)
- LocalStorage.get("/etc/passwd") resolves to /etc/passwd, ignoring base_dir
- GET /api/icons/{id}/file returns the file contents (succeeds due to LIC-02, no auth required)

Evidence

Working proof-of-concept confirmed:

```
python poc.py --target http://localhost:8000 --file /etc/passwd

[*] Target      : http://localhost:8000
[*] Server file : /etc/passwd
[1] POST /api/admin/import -> HTTP 200 {"imported":1}
[2] GET /api/icons/{id}/file -> HTTP 200
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
...
```

Files successfully exfiltrated during testing:

- /etc/passwd - confirmed
- api/dependencies.py - full authentication logic source code
- api/database.py - database schema and all queries
- data/catalog.db - complete SQLite database including personal_tokens table

The personal_tokens table schema confirmed from the exfiltrated database:

```
CREATE TABLE personal_tokens (  
    id TEXT PRIMARY KEY,  
    user_email TEXT NOT NULL UNIQUE,  
    token_hash TEXT NOT NULL,  
    expires_at TIMESTAMP NOT NULL  
);
```

On a production deployment this table contains hashed API tokens for all users.

Recommendation

Validate the path field on import against an allowlist pattern: `^icons/[a-zA-Z0-9_\-]+/[a-zA-Z0-9_\-]+\.(svg|png)$`. In `LocalStorage.get()`, resolve the full path and assert it is within `base_dir` before reading:

```
resolved = (base_dir / storage_key).resolve()  
if not resolved.is_relative_to(base_dir.resolve()):  
    raise HTTPException(400, "Invalid path")
```

LIC-04	Zip Slip in Azure Icon Import	HIGH	Open
File	scripts/import_azure_icons.py:103-106		

CVSS v3.1 Score: 7.5 - AV:N/AC:H/PR:N/UI:N/S:C/C:N/I:H/A:N

Description of the Issue

The `import_azure_icons.py` script downloads a ZIP archive from an external CDN and constructs the SVG file write path from metadata inside the archive. The category field is derived directly from `parts[-2]` of the ZIP entry name without any normalization. A ZIP entry with the structure `"Azure/Icons/./file.svg"` causes `parts[-2]` to equal `".."`, resulting in `cat_dir = icons_dir / ".."` - one directory level above the intended target directory.

```
category = parts[-2] # from zip entry path, can be ".."  
cat_dir = icons_dir / icon["category"] # no normalization  
svg_path = cat_dir / Path(icon["zip_entry"]).name  
svg_path.write_bytes(zf.read(icon["zip_entry"]))
```

This vulnerability exists specifically because the code bypasses Python's standard library extraction mechanism. The built-in **zipfile.extract()** method has automatically sanitized **../** path traversal sequences since Python 2.6, providing native protection against Zip Slip. The vulnerable code instead manually iterates over ZIP entries, reads their raw content via **zf.read()**, and constructs destination paths using **pathlib** - discarding all built-in safeguards in the process.

Impact

A malicious ZIP archive delivered via MITM attack or a compromised CDN can write files outside the **icons/Azure/** directory. Testing confirmed the traversal is limited to one directory level above **icons/Azure/** due to the **parts[-2]** parsing logic, which prevents direct overwrite of application code but enables overwriting existing icon files and injecting arbitrary content into the **icons/** directory.

Evidence

ZIP entry **"Azure/Icons/../../slip.svg"** produces **parts = ["Azure", "Icons", "..", "slip.svg"]**, so **parts[-2] = ".."** and **cat_dir = icons_dir / ".." = /app/icons/**. The file is written to **/app/icons/slip.svg** instead of **/app/icons/Azure/slip.svg**.

Recommendation

Replace the manual extraction loop with **zipfile.extract()** or **zipfile.extractall()** with an explicit path argument, which handles path sanitization natively. If manual extraction is required for filtering purposes, normalize category after extraction: **category = Path(parts[-2]).name**, and verify that the resulting **cat_dir** resolves within **icons_dir** before writing.

LIC-05	LIC-05: Insecure Outbound HTTP Request - Missing Redirect Validation and Timeout	MEDIUM	Open
File	scripts/import_azure_icons.py:50		

CVSS v3.1 Score: 7.2 - AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:L/A:N

Description of the Issue

The **import_azure_icons.py** script performs an outbound HTTP request to a hardcoded CDN URL using Python's **urllib.request.urlopen()** without configuring a timeout or limiting HTTP redirects. The destination URL is a compile-time constant - it is not derived from any user-supplied input and cannot be influenced through the API. This is therefore not an SSRF vulnerability in the standard sense, as SSRF by definition requires attacker control over the request target. The risk surface is limited to scenarios where the CDN itself or the DNS resolution path has been compromised prior to the request.

```
AZURE_ICONS_URL = "https://arch-center.azureedge.net/icons/Azure_Public_Service_Icons_V23.zip"
resp = urlopen(AZURE_ICONS_URL) # no redirect limit, no URL validation
```

Note: **AZURE_ICONS_URL** is hardcoded in the script, which prevents an attacker from directly controlling the destination URL through the API. SSRF exploitation requires a MITM attack on the CDN connection or DNS/CDN compromise.

Impact

In a DNS hijacking or CDN compromise scenario, an attacker could serve a malicious response or initiate a redirect chain targeting internal infrastructure - including the Azure instance metadata endpoint at 169.254.169.254. Additionally, the absence of a connection timeout means a slow or unresponsive CDN will block the worker thread indefinitely, creating a denial-of-service condition against the endpoint that triggers this operation. Exploiting either vector requires prior compromise of network infrastructure and is not achievable through the application API alone.

Evidence

Confirmed that the endpoint is triggerable via HTTP without code modification:

```
curl -s -X POST http://localhost:8000/api/admin/refresh-azure-icons  
{  
  "status": "started",  
  "message": "Azure icons refresh started in background"  
}
```

Recommendation

Add a redirect limit and timeout using requests with `allow_redirects=False` and manual location validation. Implement an allowlist of permitted domains for outbound HTTP requests. Consider moving the import operation out of the HTTP API surface entirely - to a cron job or CI/CD pipeline step - to eliminate the network-accessible trigger.

LIC-06	Stored XSS via Extension-Only File Type Validation	MEDIUM	Open
File	api/routers/icons.py:105		

CVSS v3.1 Score: 6.1 - AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N

RESULT: VULNERABLE - malicious SVG payload executed by browser

Description of the Issue

File type validation during upload checks only the file extension, without verifying actual file contents (magic bytes). An SVG file containing malicious JavaScript is accepted, stored, and served with Content-Type: image/svg+xml - causing the browser to execute the embedded script when the file URL is opened directly. The LIC-02 finding means the resulting XSS URL requires no authentication to access.

```
if not file.filename.endswith((".svg", ".png")):  
    raise HTTPException(400, "Only SVG and PNG files are accepted")
```

Impact

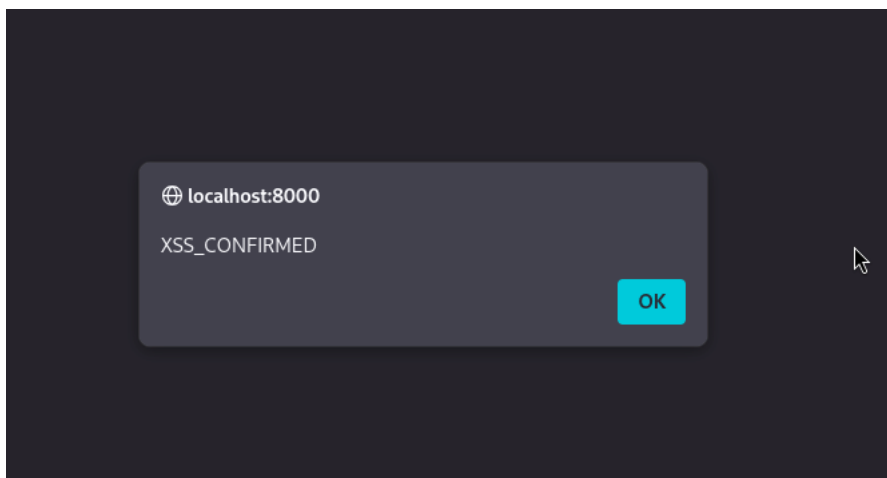
An authenticated attacker can store a malicious SVG in the application's file storage and obtain a URL that executes JavaScript in the victim's browser. Possible vectors include session token theft, API key exfiltration, phishing, and defacement. Because the file serving endpoint is public (LIC-02), the XSS URL can be delivered to any user without requiring the victim to be authenticated.

Evidence

Malicious SVG uploaded and confirmed executed in Chrome:

```
<svg xmlns="http://www.w3.org/2000/svg" onload="alert('XSS_CONFIRMED')">
  <rect width="100" height="100"/>
</svg>
```

```
curl -sv http://localhost:8000/api/icons/test_test_xss_test/file 2>&1 | grep content-type
< content-type: image/svg+xml
```



Recommendation

Verify the magic bytes of uploaded files rather than relying solely on the file extension. For SVG files, parse the content as XML and verify it contains an svg root element. Sanitize SVG content by stripping event handlers (onload, onclick, etc.) and script elements before storage. Consider using the defusedxml library for SVG parsing to prevent XXE attacks simultaneously.

3. CONCLUSION

The security assessment of the Lingaro Icons Catalog identified critical vulnerabilities across authentication configuration, access control, and file handling. The combination of fail-open authentication (LIC-01), an unauthenticated file serving endpoint (LIC-02), and unsanitized path handling in the import pipeline (LIC-03) creates a fully unauthenticated arbitrary file read chain with a CVSS score of 10.0.

During testing, the following were successfully exfiltrated without any credentials: full application source code, the complete SQLite database including the personal_tokens table containing hashed API tokens for all users, and arbitrary system files. A working Stored XSS payload was also confirmed, exploitable without authentication due to the public file serving endpoint.

The full attack chain is automated in the accompanying poc.py script, which requires only network access to the application and operates without credentials in the default configuration.

The staging environment confirms that the vulnerabilities are present in the current deployment and would be exploitable in any production instance sharing the same configuration defaults.

4. RECOMMENDATIONS

Immediate Remediation (24-48 Hours)

- LIC-01: Implement fail-safe startup validation - the server must refuse to start if authentication credentials are absent; replace the warning log with a fatal exception
- LIC-02: Add `user=Depends(require_auth)` to the `GET /{icon_id}/file` endpoint, consistent with all other icon endpoints
- LIC-03: Validate the path field on import against an allowlist pattern; in `LocalStorage.get()`, resolve the full path and assert it is within `base_dir` before reading

Short-Term Remediation (1 Week)

- LIC-04: Normalize category after extraction from ZIP metadata; verify `cat_dir` is within `icons_dir` before writing
- LIC-05: Add redirect limits and timeouts to `urlopen`; remove the import operation from the HTTP API surface
- LIC-06: Verify magic bytes of uploaded files; sanitize SVG content to strip event handlers and script elements before storage

Long-Term Remediation (1 Month)

- Architectural: Conduct a full review of all router endpoints to verify consistent application of authentication dependencies
- Architectural: Implement a centralized path sanitization mechanism in `LocalStorage` rather than relying on per-endpoint validation